

Week 5 Lecture Plan

java.util.concurrent: Bounded Resources, Pools, and Tradeoffs

Instructor: Dr. Mohamad Aoude

Pre-Class Preparation

Essential materials to prepare:

- ☐ Week 5 lab compiles: `week5-lab/week5-day1-juc-pools-backpressure`
- ☐ Smoke test passes: `gradlew.bat test -tests "edu.lu.concurrency.week5.day1.DemoSmokeTest"`
- ☐ Lecture PDFs ready: `Lecture_05_01_nar.pdf` through `Lecture_05_05_nar.pdf`
- ☐ Lab manual ready: `Lab_Week05_JavaUtilConcurrent.pdf`
- ☐ Terminal ready for live demos: queue saturation, rejected task, fixed vs cached pool, CAS counter, writer delay, `ThreadLocal` retention
- ☐ Backup screenshots or captured outputs for Demo06, Demo13, Demo16, Demo18, and Demo19
- ☐ Checkpoint evidence files ready: `CHECKPOINT2_RUBRIC.md` and `MEASUREMENT_TEMPLATE.md`

Room setup:

- Projector with slides and terminal side-by-side
- One terminal in the Week 5 lab folder
- One editor window showing only demo and exercise packages
- Keep `CONCURRENCY_SCORECARD.md`, `CHECKPOINT2_RUBRIC.md`, `MEASUREMENT_TEMPLATE.md`, `EXERCISES.md`, and `PITFALLS.md` visible for references

Learning Targets

By the end of Week 5, students must be able to:

1. Explain why unbounded queues and unbounded thread creation are production failures waiting to happen.
2. Use `ExecutorService`, `Callable`, and `Future` with correct shutdown and result handling.
3. Compare fixed and cached thread pools using observed throughput and resource behavior.
4. Choose a rejection policy based on the business meaning of overload.
5. Use atomics, read-write locks, and `ThreadLocal` only when their tradeoffs are explicit.
6. Connect Week 5 lab exercises to the Checkpoint 2 rubric and thread-pooled server evidence.

Assessment checkpoint: Exit ticket, 5 questions, 2 minutes.

Lecture Structure

Segment	Material	Goal
Part 1	Lecture_05_01_nar	Task execution, <code>ExecutorService</code> , <code>Callable</code> , <code>Future</code> , shutdown discipline
Part 2	Lecture_05_02_nar	<code>BlockingQueue</code> , producer-consumer, bounded queues, backpressure, poison pill shutdown
Part 3	Lecture_05_03_nar	Fixed vs cached pools, <code>ThreadPoolExecutor</code> , queue choice, rejection policies, pool sizing
Part 4	Lecture_05_04_nar	Measurement, atomics/CAS, read-write locks, writer delay, trade-offs
Part 5	Lecture_05_05_nar	<code>ThreadLocal</code> cleanup, measured retention, and Checkpoint 2 bridge

Recommended Time Budget

Time	Activity	Notes
0–10 min	Opening failure story	Unbounded threads or unbounded queue under load
10–35 min	Part 1 slides and Demo14	Focus on task submission vs task completion
35–60 min	Part 2 slides and Demo01–Demo03	Make backpressure visible
60–85 min	Part 3 slides and Demo06/Demo16	Use rejection matrix and measurement template
85–105 min	Part 4 slides and Demo07–Demo11/Demo18	Correctness, CAS, read throughput, and writer liveness
105–115 min	Part 5 slides and Demo12–Demo13/Demo19	<code>ThreadLocal</code> cleanup and measured retention
115–120 min	Checkpoint bridge and exit ticket	Connect exercises to rubric evidence

Live Demo Checklist

- ☐ **Demo01–Demo03:** Show queue behavior: unbounded growth, bounded rejection, blocking producer-consumer.
- ☐ **Demo06:** Show fail-fast rejection with `AbortPolicy`; explain the other policies as decision options.
- ☐ **Demo14:** Show `Callable`, `Future.get()`, timeout-aware result retrieval, and exception handling.
- ☐ **Demo16:** Run fixed vs cached comparison; record observations using `MEASUREMENT_TEMPLATE.md`.
- ☐ **Demo07–Demo09:** Contrast synchronized counter, atomic counter, and explicit CAS loop.
- ☐ **Demo10–Demo11:** Compare synchronized cache with read-write lock cache; ask what happens to writers.
- ☐ **Demo18:** Show writer delay under read pressure and discuss fair vs unfair locks.
- ☐ **Demo12–Demo13:** Show `ThreadLocal` context and stale value leak in pooled workers.
- ☐ **Demo19:** Show retained bytes with and without `ThreadLocal` cleanup.

Rejection Policy Discussion

Policy	Business meaning
AbortPolicy	Fail fast; caller must observe overload.
CallerRunsPolicy	Slow the producer; useful backpressure when caller can safely do the work.
DiscardPolicy	Drop only optional work; never silently drop required requests.
DiscardOldestPolicy	Prefer fresh work over stale queued work.

Student Work Contract

Students implement only:

- Ex1_BoundedBuffer
- Ex2_BoundedExecutor
- Ex3_AtomicMetrics
- Ex4_ReadMostlyCache
- Ex5_RequestContext

They must run:

```
.\gradlew.bat studentCheck
.\gradlew.bat test --tests "edu.lu.concurrency.week5.day1.DemoSmokeTest"
```

They must also submit:

- completed CONCURRENTCY_SCORECARD.md
- measurement table using MEASUREMENT_TEMPLATE.md
- short tradeoff paragraph using symptom, mechanism, fix, and remaining risk

Checkpoint 2 Bridge

Checkpoint 2 is a thread-pooled server with bounded resources and fairness notes.

- Ex1 maps to the request queue.
- Ex2 maps to the bounded execution policy.
- Ex3 maps to request metrics and latency accounting.
- Ex4 maps to read-heavy shared state or cache behavior.
- Ex5 maps to per-request context isolation and cleanup.

Rubric weights:

Area	Weight
Bounded request queue	20%
Bounded executor policy	20%
Correctness under contention	20%
Liveness and fairness notes	15%
Measurement evidence	15%
Request context cleanup	10%

Exit Ticket

1. Why is `newCachedThreadPool()` dangerous under sustained load?
2. What is the difference between blocking, rejecting, and dropping work?
3. When should `Future.get(long timeout, TimeUnit unit)` be preferred over plain `get()`?
4. Why can `ReentrantReadWriteLock` improve throughput but still create a liveness problem?
5. What does Demo19 measure, and what must always happen after using `ThreadLocal` in a pooled worker?

Instructor Notes

- Keep `Demo17_HandRolledExecutorContrast` as an instructor discussion only. It is a contrast example, not a student pattern to copy.
- Mention Release 2 pitfalls: oversized pools, unbounded queues, and nested task submission risks.
- Do not promise stable benchmark numbers. Require methodology and explanation, not a universal speedup.
- Treat Demo18 and Demo19 as evidence demos: they make liveness and retention risks visible without becoming student exercise requirements.
- Keep the Checkpoint 2 rubric visible while explaining Ex1–Ex5; students should know exactly what evidence they must produce.
- Close with the Week 6 handoff: Fork/Join asks a different question: how large must CPU-bound work be before decomposition pays for itself?